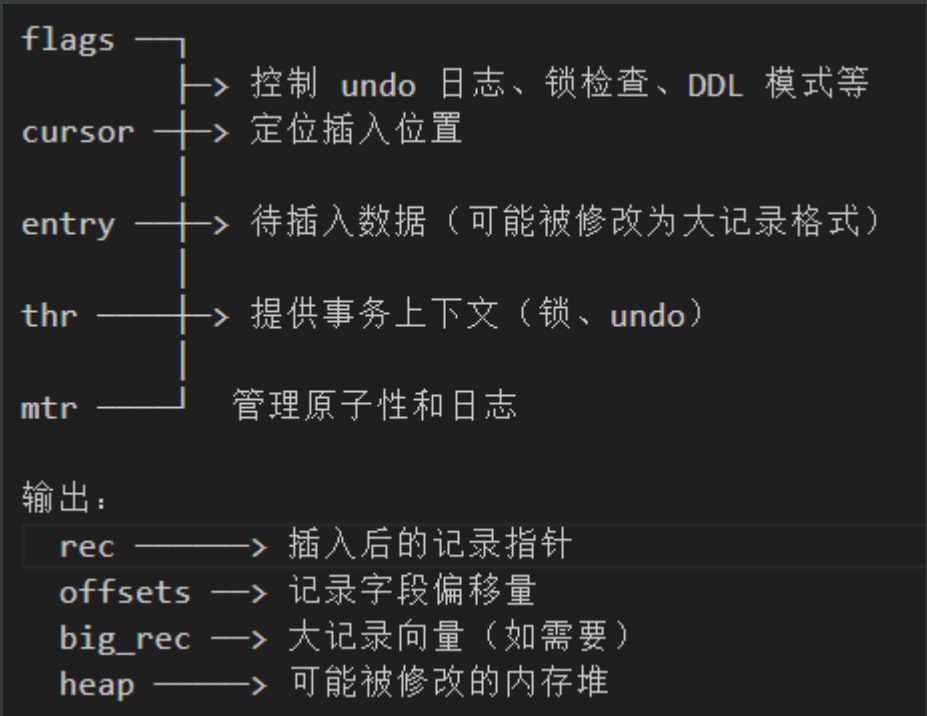


1、悲观写

1.1 总览

1.1.1 基本信息

函数名称: `btr_cur_pessimistic_insert`
核心作用: 在 B+ 树索引中执行悲观插入, 处理页面分裂、大记录、锁检查、undo 日志等。当乐观插入失败 (如页面空间不足) 时调用。
所属模块: `mysql/storage/innobase/btr0cur.cc` 随后搜索函数 `dberr_t btr_cur_pessimistic_insert`
参数分析:



树状流程图:

btr_cur_pessimistic_insert

- ├ 入口与准备
 - ├ 校验 entry 类型、mtr 持锁、标志合法
 - └ 初始化 big_rec=null, cursor->flag=BTR_CUR_BINARY
- ├ 1) 锁与 Undo
 - ├ btr_cur_ins_lock_and_undo: 检查锁冲突, 写聚集索引 undo, 回填 roll_ptr
 - └ 失败 → 返回错误
- ├ 2) 预留空间 (如需)
 - ├ 条件: 未设 BTR_NO_UNDO_LOG_FLAG 或表是 intrinsic
 - ├ 计算 n_extents = tree_height/16 + 3
 - └ fsp_reserve_free_extents 预留; 失败 → DB_OUT_OF_FILE_SPACE
- ├ 3) 大记录判断/拆分
 - ├ page_zip_rec_needs_ext 判定是否页内放不下
 - ├ 需外存 → dtuple_convert_big_rec 拆大字段, 得 big_rec_vec
 - └ 拆失败 → 释放预留区, 返回 DB_TOO_BIG_RECORD
- ├ 4) 插入动作
 - ├ 若当前页是根 → btr_root_raise_and_insert (可能升高树)
 - ├ 否则 → btr_page_split_and_insert (必要时分裂)
 - └ *rec 为空 → DB_OUT_OF_FILE_SPACE
- ├ 5) 插后锁处理 (未禁锁)
 - ├ 空间索引: 跳过
 - ├ 非聚集: page_update_max_trx_id
 - └ 若非 infimum 或无前页 → inherit=true (需要锁继承)
- ├ 6) AHI 与锁继承
 - ├ 未禁 AHI → btr_search_update_hash_on_insert
 - └ inherit 且未禁锁 → lock_update_insert
- ├ 7) 收尾
 - ├ 若预留区 n_reserved>0 → fil_space_release_free_extents
 - ├ big_rec 输出 big_rec_vec (调用者去写 LOB)
 - └ 返回 DB_SUCCESS

1.1.2 思想

悲观写的整体思想:

- 先拿住必要的锁和资源, 再动手改页, 保证“要么能顺利插入, 要么提前等/失败”, 不把冲突留到后面再回滚。
- 避免并发冲突: 在修改前检查并获取记录/GAP 锁; 必要时等待, 避免

写到一半才发现冲突。

- 避免空间不足等半途而废：预留页/区、分裂时也确保可完成。
- 日志先行：在修改前准备好 undo（可回滚、供 MVCC），redo 由 mtr 收集，保证崩溃恢复。

1.3 函数逻辑详解（重点）

1.3.1 锁检查和 Undo 日志写入（2969-2975行）

```
/*
```

锁序列：先做锁检查并加锁（普通索引 **GAP**/记录锁或空间索引谓词锁），如果冲突则在此处等待/返回，不进入后续页分裂/写页。

可回滚：若是聚集索引插入，先写一条 **undo**，并把 **roll_ptr** 回填到要插的记录里，为回滚和 **MVCC** 留好退路。

输出 **inherit**：告诉后续是否需要在新行位置继承 **GAP** 锁，保持范围锁一致性。

```
*/
```

```
err = btr_cur_ins_lock_and_undo(flags, cursor, entry, thr,  
mtr, &inherit);
```

```
    if (err != DB_SUCCESS) {  
        return (err);  
    }
```

作用：检查锁冲突（GAP 锁、谓词锁等）；对聚集索引写入 undo 日志；设置回滚指针（roll_ptr）

1.3.2 预留文件空间（2977-2989行）

```
/*
    只有当需要写undo（flag为0）或表为intrinsic时才需要分配空间
    根据树高计算需要预留的区数，与表空间id一并传入fsp，然后输出预留
    空间n_reserved
    若success为false，则预留失败
*/

if (!(flags & BTR_NO_UNDO_LOG_FLAG) || index->table-
>is_intrinsic()) {
    uint n_extents = cursor->tree_height / 16 + 3;
    success = fsp_reserve_free_extents(&n_reserved, index-
>space, n_extents,
                                     FSP_NORMAL, mtr);

    if (!success) {
        return (DB_OUT_OF_FILE_SPACE);
    }
}
```

- 预留空间： `n_extents = tree_height / 16 + 3`，防止分裂时空间不足
- 失败时返回 `DB_OUT_OF_FILE_SPACE`

1.3.3 处理大记录（2991-3013行）

先判断记录是否过大、需外部存储（LOB/off-page）；在插入前就把大字段拆出去，让页内能放下。

转换失败直接返回 DB_TOO_BIG_RECORD，并释放前面预留的区，避免做到一半才发现放不下。

转换成功则返回 big_rec_vec 让调用者后续把大字段写到外部页，确保插入路径可继续。

```
// 分别传入记录需要的索引数、表的格式、记录的列数、表的页大小
if (page_zip_rec_needs_ext(rec_get_converted_size(index,
entry),
                                dict_table_is_comp(index->table),
                                dtuple_get_n_fields(entry),
                                dict_table_page_size(index-
>table))) {
    // 防御性编程,一般不会发生
    if (UNIV_LIKELY_NULL(big_rec_vec)) {
        dtuple_convert_back_big_rec(entry, big_rec_vec);
        ut_d(ut_error);
    }
    // 大记录拆分
    big_rec_vec = dtuple_convert_big_rec(index, nullptr,
entry);

    if (big_rec_vec == nullptr) {
        if (n_reserved > 0) {
```

```

        fil_space_release_free_extents(index->space,
n_reserved);
    }
    return (DB_TOO_BIG_RECORD);
}
}

```

- 判断是否需要外部存储 (LOB)
- 转换大记录, 失败时释放预留空间并返回 `DB_TOO_BIG_RECORD`

1.3.4 执行插入 (3015-3025行)

```

if (dict_index_get_page(index) ==
    btr_cur_get_block(cursor)->page.id.page_no()) {
    /* The page is the root page */
    *rec = btr_root_raise_and_insert(flags, cursor, offsets,
heap, entry, mtr);
} else {
    *rec = btr_page_split_and_insert(flags, cursor, offsets,
heap, entry, mtr);
}

if (!*rec) {
    return DB_OUT_OF_FILE_SPACE;
}

```

- 根页面: 调用 `btr_root_raise_and_insert` (可能提升树高度)

- 非根页面：调用 `btr_page_split_and_insert`（页面分裂）
- 插入失败返回 `DB_OUT_OF_FILE_SPACE`

1.3.5 更新锁和哈希索引（3030-3058行）

```
// 若flags = ..10,即不做锁处理,就不会进入if
if (!(flags & BTR_NO_LOCKING_FLAG)) {
    ut_ad(!index->table->is_temporary());
    if (dict_index_is_spatial(index)) {
        // 若表为临时表,则不走这一套锁逻辑
    } else {
        // 若不为聚类索引,则要更新页上的max_trx_id
        if (!index->is_clustered()) {
            page_update_max_trx_id(btr_cur_get_block(cursor),
                                   btr_cur_get_page_zip(cursor),
                                   thr_get_trx(thr)->thr, mtr);
        }
        //
        if (!page_rec_is_infimum(btr_cur_get_rec(cursor)) ||

btr_page_get_prev(buf_block_get_frame(btr_cur_get_block(cursor)),

                                   mtr) == FIL_NULL) {
            // inherit设为true,目的是告诉后续流程需要做锁继承
            inherit = true;
        }
    }
}
```

```

}

if (!index->disable_ahi) {
    btr_search_update_hash_on_insert(cursor);
}

if (inherit && !(flags & BTR_NO_LOCKING_FLAG)) {
    lock_update_insert(btr_cur_get_block(cursor), *rec);
}

```

- 更新非聚集索引页的最大事务 ID
- 更新自适应哈希索引 (AHI)
- 处理锁继承 (GAP 锁)

1.3.6 资源清理 (3060-3066行)

```

if (n_reserved > 0) {
    fil_space_release_free_extents(index->space, n_reserved);
}

*big_rec = big_rec_vec;

return (DB_SUCCESS);

```

- 释放预留的文件空间
- 返回大记录向量给调用者

2、悲观更新

2.1 总览

2.1.1 基本信息

函数名称: `btr_cur_pessimistic_update`

核心作用: 用于在 B+ 树索引中更新记录, 当乐观更新失败 (空间不足、压缩溢出等) 时调用。

所属模块: `mysql/storage/innobase/btr0cur.cc` 随后搜索函数 `dberr_t`

`btr_cur_pessimistic_update`

树状流程图:

入口：先调乐观更新 `btr_cur_optimistic_update`

- └ 返回 `DB_SUCCESS` → 直接返回成功
- └ 返回 `OVERFLOW/UNDERFLOW/ZIP_OVERFLOW` → 进入悲观路径
- └ 其他错误 → 清理后返回错误

悲观路径：

- 1) 取旧记录 `rec`，算 `offsets`，拷贝成 `new_entry`
- 2) 把更新值写进 `new_entry`，处理默认列
- 3) 判断新记录是否过大，需外部存储？
 - └ 需 → 拆大字段，生成 `big_rec_vec`；失败则释放预留并报“记录太大”
 - └ 不需 → 继续
- 4) 锁检查 + 写 `undo`，拿 `roll_ptr`；失败直接返回
- 5) 若乐观结果是 `OVERFLOW` → 预留表空间区 (`n_extents`)
- 6) 填系统字段：`roll_ptr / trx_id` 到 `new_entry`
- 7) 预存锁到页 `infimum`，更新哈希（删除旧记录）
- 8) 删除旧记录，尝试“直接插入”新记录
 - └ 成功：
 - 游标指向新记录，锁搬回新记录
 - 外部字段归属标记
 - 尝试压缩页；否则更新插入缓冲位图
 - 可能释放索引锁
 - 设置成功，跳收尾
 - └ 失败：
 - 重置插入缓冲位图（如需）
 - 若有大字段，确保索引锁持有
 - 记是否是页首记录
 - 调悲观插入 `btr_cur_pessimistic_insert`（禁 `undo`/锁）
 - 补 `PAGE_MAX_TRX_ID`（非聚集/非临时）
 - 外部字段归属标记
 - 锁从 `infimum` 搬回，必要时修复 `supremum` 锁
 - 继续收尾

收尾：

- 若预留了区 → 释放
- 返回 `big_rec_vec` 给调用者（去写 `LOB`）
- 返回 `err`（成功或前面错误）

2.1.2 思想

- 先试乐观（原地改），失败再走悲观；悲观路径是“删旧+插新”的完整流程。

- 先拿锁/写 undo、必要时预留空间，确保要么能做完，要么尽早失败，不留半成品。
- 处理大记录：页放不下就拆到页外；不行就直接报“记录过大”。
- 真正更新时删除旧记录、插入新记录，必要时分裂/重组页面。
- 插后补齐元信息：锁继承、事务 ID、插入缓冲位图、自适应哈希等。
- 资源善后：失败释放预留区、清理 big_rec；成功返回 big_rec 给调用者写 LOB。

2.2 函数逻辑详解（重点）

2.2.1 尝试乐观更新（3819~3821）

```
// 先进行乐观更新,结果保存到err中
err = optim_err = btr_cur_optimistic_update(
    flags | BTR_KEEP_IBUF_BITMAP, cursor, offsets,
    offsets_heap, update,
    cpl_info, thr, trx_id, mtr);
```

2.2.2 处理乐观更新结果 (3823~3845)

```
switch (err) {  
    // 这三种属于页空间/压缩相关的问题,继续走悲观更新流程  
    case DB_ZIP_OVERFLOW: // 压缩页更新后“压缩不下”溢出。  
    case DB_UNDERFLOW: // 页过空（删除/移动导致不够满）。  
    case DB_OVERFLOW: // 页溢出（空间不足）。  
        break;  
    default:  
        goto err_exit; // 其他错误或DB_SUCCESS,做资源清理,然后  
return err  
}
```

2.2.3 构建新记录 (3847~3866)

当我们想修改一条记录时，理想的情况是原地改，但如果页空间不足，原地改不了，就要换一种做法，即把旧记录删除，然后插入一条新纪录

```
rec = btr_cur_get_rec(cursor); // 拿到cursor指向的旧记录指针  
*offsets = rec_get_offsets(rec, index, *offsets, ...); // 计算  
偏移信息  
dtuple_t *new_entry = row_rec_to_index_entry(rec, index,  
*offsets, entry_heap); // 复制一份旧记录的副本  
row_upd_index_replace_new_col_vals_index_pos(new_entry, index,  
update, false, entry_heap); // 把update里的新值写入new_entry对应  
列,完成新纪录构建
```

2.2.4 处理大记录 (3874~3897)

```
// 先判断是否为大记录
if (page_zip_rec_needs_ext(rec_get_converted_size(index,
new_entry),
                                page_is_comp(page),
dict_index_get_n_fields(index),
                                block->page.size)) {
    // 为大记录,需要拆分,big_rec_vec记录哪些字段要放到页外
    big_rec_vec = dtuple_convert_big_rec(index, update,
new_entry);
    // 如果big_rec_vec为空,说明拆分失败
    if (UNIV_UNLIKELY(big_rec_vec == nullptr)) {
        if (n_reserved > 0) {           // 清理预留空间
            fil_space_release_free_extents(index->space,
n_reserved);
        }
        err = DB_TOO_BIG_RECORD; // 返回"记录太大的错误"
        goto err_exit;
    }
}
```

2.2.5 锁检查和Undo日志 (3899~3904)

// 检查是否有锁冲突 + 对聚类索引更新写一条undo记录,拿到roll_ptr给后面填到系统字段

```
err = btr_cur_upd_lock_and_undo(
    flags, cursor, *offsets, update, cpl_info, thr, mtr,
    &roll_ptr);
if (err != DB_SUCCESS) {
    goto err_exit;
}
```

2.2.6 预留空间 (3961~3975)

// 如果之前乐观更新失败是因为页空间不够,则需要先预留空间

```
if (optim_err == DB_OVERFLOW) {
    ulint n_extents = cursor->tree_height / 16 + 3;
    if (!fsp_reserve_free_extents(
        &n_reserved, index->space, n_extents,
        flags & BTR_NO_UNDO_LOG_FLAG ? FSP_CLEANING :
        FSP_NORMAL, mtr)) {
        // 预留失败
        err = DB_OUT_OF_FILE_SPACE;
        goto err_exit;
    }
}
```

2.2.7 删除旧记录并插入新记录 (3994~4010)

```
if (!dict_table_is_locking_disabled(index->table)) {  
    lock_rec_store_on_page_infimum(block, rec); // 先把旧记录的锁  
    挂到最小占位记录上  
}  
btr_search_update_hash_on_delete(cursor); // 更新自适应哈希索  
引,告诉哈希缓存这条旧记录要删了  
page_cursor = btr_cur_get_page_cur(cursor);  
page_cur_delete_rec(page_cursor, index, *offsets, mtr); // 删  
除当前记录  
page_cur_move_to_prev(page_cursor); // cursor移到前一条  
rec =  
    btr_cur_insert_if_possible(cursor, new_entry, offsets,  
offsets_heap, mtr); // 尝试插入
```

2.2.8 插入后 (4011~4164)

根据路径分析的内容对应看源码即可

3、悲观删除

3.1 总览

3.1.1 基本信息

函数名称: btr_cur_pessimistic_delete

核心作用: 它负责在 B+ 树索引中删除一条记录, 并处理可能带来的各种复

杂连锁反应，比如页合并、节点指针更新、外部字段释放等，保证删除操作的原子性和一致性。

所属模块：mysql/storage/innobase/btr0cur.cc 随后搜索函数bool
btr_cur_pessimistic_delete

btr_cur_pessimistic_delete

- ├─ 入口与准备
 - ├─ 校验 `flags`、`mtr` 持锁、标志合法
 - ├─ 获取 `block`、`page`、`index`、`rec`
 - └─ 计算 `offsets`
- ├─ 1) 处理外部字段 (LOB)
 - ├─ 若 `rec_offs_any_extern(offsets)` → 有外部字段
 - ├─ `btr_ctx.free_externally_stored_fields`: 释放外部字段
 - ├─ 游标可能变化, 重新获取 `cursor/block/page/rec`
 - └─ 特殊检查: `purge` 线程且记录未删除标记 → 返回成功
- ├─ 2) 预留空间 (如需)
 - ├─ 条件: `!has_reserved_extents && !rollback`
 - ├─ 计算 `n_extents = tree_height/32 + 1`
 - └─ `fsp_reserve_free_extents` 预留 (FSP_CLEANING); 失败 → DB_OUT_OF_SPACE
- ├─ 3) 特殊情况: 丢弃整页
 - ├─ 条件: 页记录数 < 2 且不是根页
 - └─ `btr_discard_page` 丢弃整页 → 跳收尾
- ├─ 4) 更新删除锁
 - └─ 若 `flags == 0` → `lock_update_delete`
- ├─ 5) 处理非叶子层最左节点
 - ├─ 若 `level > 0` 且删除的是最左节点指针
 - ├─ 若页无前页 (FIL_NULL)
 - └─ `btr_set_min_rec_mark`: 标记新的最左节点为最小记录
 - ├─ 若空间索引
 - └─ `rtr_update_mbr_field`: 更新父页的 MBR
 - └─ 否则
 - ├─ `btr_node_ptr_delete`: 删除父页的节点指针
 - └─ `btr_insert_on_non_leaf_level`: 插入新的节点指针到父页
- ├─ 6) 删除记录
 - ├─ `btr_search_update_hash_on_delete`: 更新哈希索引
 - └─ `page_cur_delete_rec`: 删除记录
- ├─ 7) 压缩页面 (如需)
 - └─ `btr_cur_compress_if_useful`: 尝试压缩页面
- └─ 8) 收尾
 - ├─ 若叶子页且非只读且非在线 DDL → 释放索引锁
 - ├─ 若预留区 `n_reserved > 0` → `fil_space_release_free_extents`
 - └─ 返回 `ret` (是否压缩成功)

3.1.2 思想

3.2 函数逻辑详解（重点）

3.2.1 准备工作（4631~4653）

```
block = btr_cur_get_block(cursor); // 拿到要删除的记录所在的页块
page = buf_block_get_frame(block); // 拿到页的内容
rec = btr_cur_get_rec(cursor); // 拿到要删除的记录
offsets = rec_get_offsets(...); // 算出记录的字段位置（方便后面操作）
```

3.2.2 处理外部字段（4655~4689）

```
// 若该字段有外部存储,需要一起删除
if (rec_offs_any_extern(offsets)) {
    // 创建上下文释放外部字段
    lob::BtrContext btr_ctx(mtr, pcur, index, rec, offsets,
block);
    btr_ctx.free_externally_stored_fields(trx_id, undo_no,
rollback, rec_type,
node);

    // 重新定位游标
    if (pcur != nullptr) {
        cursor = pcur->get_btr_cur();
        block = btr_cur_get_block(cursor);
        page = buf_block_get_frame(block);
        rec = btr_cur_get_rec(cursor);
    }
}
```

```

// 记录已经被其他事务复用了,直接返回成功
if (!rollback && rec_type != 0 &&
    !rec_get_deleted_flag(rec, rec_offs_comp(offsets))) {
    *err = DB_SUCCESS;
    return true;
}
}

```

3.2.3 预留空间 (4691~4705)

删除后可能触发更新父页指针、合并页面等操作，这些操作需要空间

```

// 未预留且非回滚情况下预留空间
if (!has_reserved_extents && !rollback) {
    ulint n_extents = cursor->tree_height / 32 + 1; // 计算需要
    预留的区数
    success = fsp_reserve_free_extents(&n_reserved, index-
    >space, n_extents,
                                     FSP_CLEANING, mtr); //

```

预留空间操作

```

    if (!success) {
        *err = DB_OUT_OF_FILE_SPACE;
        return false; // 预留失败返回错误
    }
}

```

3.2.4 丢弃整页判断（4707~4717）

如果页内只剩一条记录，应直接删除该页，然后把页空间还给表（根页不能丢弃）

```
// 若当前页记录小于2且非根页
if (UNIV_UNLIKELY(page_get_n_recs(page) < 2) &&
    UNIV_UNLIKELY(dict_index_get_page(index) != block-
>page.id.page_no())) {
    btr_discard_page(cursor, mtr); // 丢弃整页
    ret = true; // 标记整页已丢弃
    goto return_after_reservations; // 跳转到收尾代码
}
```

3.2.5 更新删除锁（4719~4721）

删除记录时，其他事务可能正在访问或等待这条记录。如果不通知它们，可能导致：

死锁：事务A等待记录X，事务B正在删除记录X，互相等待

数据不一致：事务A读取记录X，但记录X正在被删除

幻读：事务A查询时看到记录X，但记录X已被删除

在删除前更新锁信息，告知其他事务这条记录正在被删除，让它们：知道记录状态变化、决定是等待还是放弃、避免冲突和死锁

```
if (flags == 0) // flag = 0表示删除操作
    lock_update_delete(block, rec);
```

3.2.6 处理非叶子层的最左节点 (4723~4785)

若删除的是非叶节点的最左节点，根据B+Tree的特性，会使该节点的父节点指向失效，，因此需要更新父页的节点指针，使其指向新的最左节点

```
level = btr_page_get_level(page); // 获取页的层级,大于0为非叶子层

// 判断是否需要处理,条件是非页层+删除的是最左节点(如果rec=下一条记录,说明是最左节点)

if (level > 0 && UNIV_UNLIKELY(rec ==
page_rec_get_next(page_get_infimum_rec(page)))) {
    rec_t *next_rec = page_rec_get_next(rec); // 将被删除记录的
下一条记录作为新的最左节点

    /*
    下面分为三种情况：
        1. 页无前页
        2. 空间索引
        3. 普通索引
    */

    if (btr_page_get_prev(page, mtr) == FIL_NULL) {
        btr_set_min_rec_mark(next_rec, mtr);
    } else if (dict_index_is_spatial(index)) {
        rtr_mbr_t father_mbr;
        rec_t *father_rec;
        btr_cur_t father_cursor;
        ulint *offsets;
        bool upd_ret;
        ulint len;
```



```
}  
}
```

3.2.7 删除记录 (4787~4789)

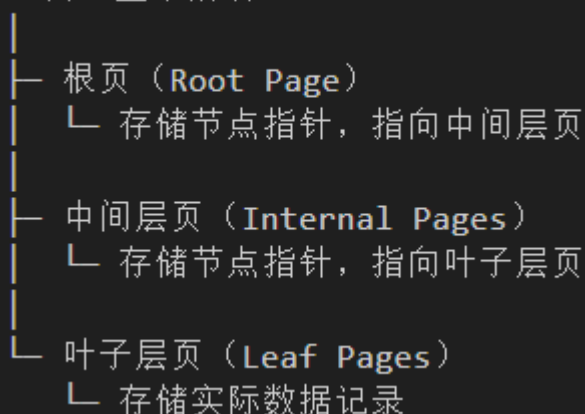
3.2.8 压缩页面 (4801~4803)

3.2.9 收尾 (4798~4820)

4、B+树在InnoDB中的存储结构(重要)

4.1 整体结构：B+树 = 多个页组成的树

B+树（整个索引）



- B+ 树由多个页组成, 每个页是 16KB 的固定大小
- 页之间通过指针连接, 形成树形结构

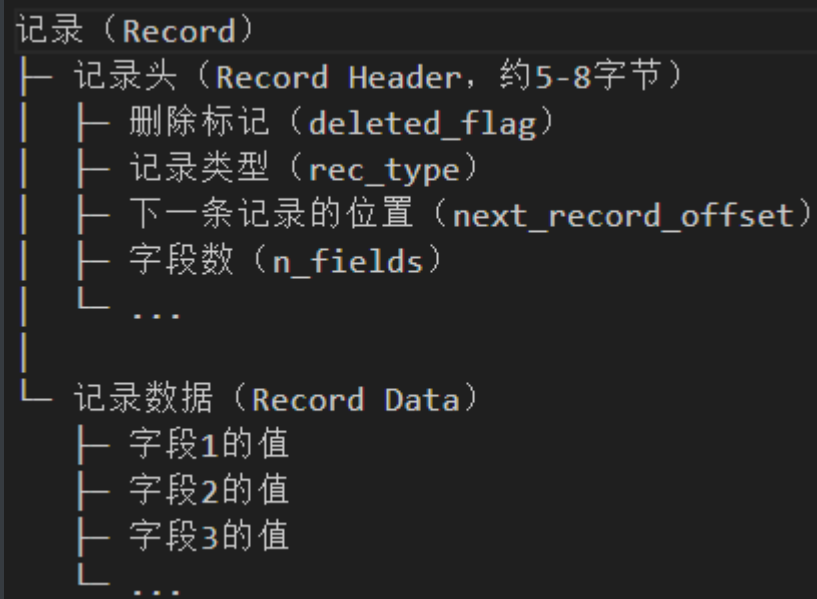
4.2 页的存储：每个页都有固定格式



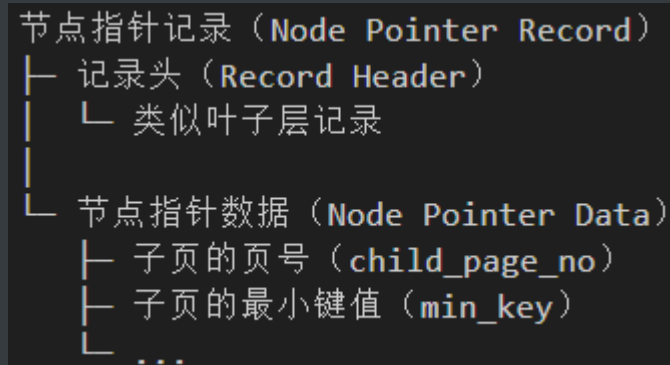
- 页头：页的元数据（页号、类型、记录数等）
- 记录区域：实际数据（记录按顺序存储）
- 页目录：槽数组，每个槽指向一条记录的起始位置（用于快速定位）
- 页尾：校验和等

4.3 记录的存储：每条记录都有固定格式

4.3.1 叶子层记录（存储实际数据）



4.3.2 非叶子层记录（存储节点指针）



4.4 它们之间的关系

4.4.1 层级关系

```
层级0（叶子层）：  
页1 → 记录1，记录2，记录3  
页2 → 记录4，记录5，记录6  
页3 → 记录7，记录8，记录9  
  
层级1（中间层）：  
页4 → 节点指针1（指向页1，键值=记录1的键值）  
      节点指针2（指向页2，键值=记录4的键值）  
      节点指针3（指向页3，键值=记录7的键值）  
  
层级2（根页）：  
页5 → 节点指针1（指向页4，键值=记录1的键值）
```

- 根页：树的顶层，只有一个
- 中间层：存储节点指针，指向下一层
- 叶子层：存储实际数据记录

4.4.2 页之间的连接

```
页1 ↔ 页2 ↔ 页3  
（通过页头中的 prev_page 和 next_page 连接）
```

- 同一层的页通过双向链表连接
- 页头中有 FIL_PAGE_PREV 和 FIL_PAGE_NEXT 字段

4.5 存储的物理位置

4.5.1 表空间的存储

```
表空间 (Tablespace)
├─ 页0: 文件头页
├─ 页1: 扩展描述页
├─ 页2: 根页 (B+树的根)
├─ 页3: 中间层页1
├─ 页4: 中间层页2
├─ 页5: 叶子层页1
├─ 页6: 叶子层页2
└─ ...
```

- 每个页都有一个页号 (page_no) , 用于定位
- 页号是连续的, 但逻辑上通过指针连接成树

4.5.2 页在内存中的存储

```
磁盘上的页 (16KB)
↓ 读入内存
缓冲池 (Buffer Pool) 中的页块 (buf_block_t)
↓ 通过指针访问
页内容 (page_t)
↓ 通过 rec 指针定位
记录 (rec_t)
```

- 磁盘上的页读入内存后, 存储在缓冲池中
- 通过 buf_block_t 访问页, 通过 rec_t 指针访问记录

4.6 记录的定位方式

4.6.1 通过页目录定位

```
要查找记录3:
1. 通过页目录找到槽2
2. 槽2指向记录3的位置
3. 通过 rec 指针访问记录3
```

- 页目录提供快速定位：通过槽号找到记录位置
- `rec` 指针：指向记录在页内的起始位置

4.6.2 通过偏移量访问字段

```
要访问记录3的字段2：  
1. rec 指向记录3的起始位置  
2. offsets[1] 是字段2的偏移量  
3. rec + offsets[1] 就是字段2的位置
```

- `offsets` 数组：记录每个字段相对于记录起始位置的偏移量
- 通过 `rec + offsets[i]` 访问第 `i` 个字段

4.7 总结

- B+ 树由多个页组成，每个页 16KB，有固定格式
- 页包含页头、记录区域、页目录、页尾
- 记录包含记录头和记录数据
- 叶子层存储实际数据，非叶子层存储节点指针
- 页之间通过指针连接，形成树形结构
- 记录通过页目录和 `rec` 指针定位，字段通过 `offsets` 数组访问